

Author: Software Engineering Team Member

Project: Task Manager Application (Python 3.11 Implementation)

Date: July 21, 2026

Task Manager Codebase Exploration & Discovery Report

1. Initial vs. Final Understanding of the Codebase

Initial Observations (Pre-Analysis)

- **Directory Structure & Tech Stack:** Standard Python project requiring Python 3.11+ with zero external third-party dependencies (strictly standard library).
- **Entry Point:** cli.py acts as the command-line entry point using argparse or similar standard CLI parsing.
- **Architecture Hypothesis:** A simple layered architecture consisting of:
 - CLI presentation layer (cli.py)
 - Data persistence layer (JSON file or local storage)
 - Domain models and core task management logic
 - Test suite directory (tests/) running via standard unittest.

Refined Understanding (Post-Analysis)

- **Architecture:** The application follows a clean Separation of Concerns (SoC) design:
 - **CLI / Presentation Layer (cli.py):** Responsible for command parsing, validating flags, formatting CLI output, and routing user actions to the manager layer.
 - **Service / Management Layer (task_manager.py):** Handles task collection operations, filtering, statistics calculations, and business rule enforcement.
 - **Domain Layer (models.py / task.py):** Encapsulates core entities (Task, TaskStatus, TaskPriority) and field validations.
 - **Persistence Layer (repository.py or JSON storage module):** Encapsulates loading and saving task state to disk using standard library modules (json, pathlib).
- **Key Misconception Cleared:** Initially assumed business logic might be embedded directly inside cli.py. In reality, cli.py is thin, deferring business decisions and task filtering to dedicated service classes.

2. Key Insights Gained from AI Prompts

Part 1: Project Structure & Stack Insights

- **Prompt Strategy:** Querying the AI for structural entry points and dependencies.
- **Key Finding:** Identified that the project intentionally relies 100% on Python's built-in libraries (datetime, json, argparse, dataclasses, enum).
- **Architectural Flow:** Command Invocation (cli.py) → Repository/Storage → Domain Processing (Task) → Response/View Output.

Part 2: Feature Location (CSV Export)

- **Prompt Strategy:** Locating existing data serialization patterns to implement Task Export to CSV.
- **Key Finding:** Existing persistence uses JSON serialization in the repository layer.
- **Implementation Plan for CSV Export:**
 - Add a CSVExporter utility or method in a dedicated exporters.py module (or inside task_manager.py / repository.py).
 - Utilize Python's built-in csv module (csv.DictWriter).
 - Extend cli.py with an --export csv flag under the list command or create a standalone python cli.py export --format csv command.

Part 3: Domain Model & Business Rules

- **Entities:**
 - Task: Core dataclass/class containing id, title, description, priority, status, due_date, tags, and created_at.
 - TaskStatus (Enum): TODO, IN_PROGRESS, REVIEW, DONE, (and candidate state ABANDONED).
 - TaskPriority (Enum/Int): 1=LOW, 2=MEDIUM, 3=HIGH, 4=URGENT.
- **Domain Glossary:**
 - **Overdue:** Current date is past due_date AND status is not DONE.
 - **Abandoned:** Status assigned to stale overdue tasks under specific business rules.

3. Practical Application: Overdue Abandonment Business Rule

"Tasks that are overdue for more than 7 days should be automatically marked as abandoned unless they are marked as high priority (Priority 3 or 4)."

Affected Files & Modifications

1. **models.py / task.py:**

- Add ABANDONED = "abandoned" to the TaskStatus enum.

2. **task_manager.py (Business Logic):**

- Add a method process_overdue_abandonment():

```
from datetime import datetime, timedelta
```

```
def process_overdue_abandonment(self) -> list[Task]:
    now = datetime.now()
    abandoned_tasks = []
    for task in self.get_all_tasks():
        if task.status != TaskStatus.DONE and task.due_date:
            days_overdue = (now - task.due_date).days
            # Check rule: > 7 days overdue AND priority < HIGH (3)
            if days_overdue > 7 and task.priority < TaskPriority.HIGH:
                task.status = TaskStatus.ABANDONED
                abandoned_tasks.append(task)
    if abandoned_tasks:
        self.repository.save_all()
    return abandoned_tasks
```

3. **cli.py (CLI Interface):**

- Add a command option (e.g., python cli.py maintenance cleanup-overdue) or automatically run this check when invoking python cli.py list --overdue or python cli.py stats.

4. **tests/test_task_manager.py:**

- Add unit tests covering edge cases:
 - Task 6 days overdue → Unchanged.
 - Task 8 days overdue, Priority 2 (Medium) → Marked ABANDONED.
 - Task 8 days overdue, Priority 3 (High) → Unchanged (Protected).
 - Completed task (DONE) 10 days past due → Unchanged.

Questions for the Team Before Implementation

1. Should ABANDONED tasks be hidden from standard python cli.py list calls by default?
2. Is abandonment triggered automatically on every CLI command execution, or via a scheduled maintenance CLI command?
3. Are URGENT (Priority 4) tasks also exempt alongside HIGH (Priority 3) tasks? (*Assumed exempt based on priority threshold ≥ 3*).

4. Future Strategies for Onboarding Unfamiliar Codebases

1. **Top-Down & Bottom-Up Hybrid Exploration:**

- Start at entry points (cli.py, main.py, index.js) to see how inputs enter the app.
- Examine core domain models (models.py) to understand the underlying data

structures.

2. Leverage Tests as Documentation:

- Unit tests (tests/) serve as executable specifications showing exactly how functions are intended to be called and how edge cases are handled.

3. AI-Assisted Context Framing:

- Use targeted prompts asking AI to explain file interactions, data flow, and hidden architectural patterns rather than asking for broad, generic code overviews.